

HOMework 6

Kavya Saranya K V
2025115152

1. Simulate the undo and redo option in MS-Word packages using a doubly linked list.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    char text[100];
    struct Node *prev, *next;
} Node;

Node *current = NULL;

Node* newNode(char *t) {
    Node *n = (Node*)malloc(sizeof(Node));
    strcpy(n->text, t);
    n->prev = n->next = NULL;
    return n;
}

void freeForward(Node *n) {
    while (n) {
        Node *t = n->next;
        free(n);
        n = t;
    }
}

void doAction(char *t) {
    Node *n = newNode(t);
    if (current) {
        freeForward(current->next);
        current->next = n;
        n->prev = current;
    }
    current = n;
}

void undo() {
    if (current->prev)
        current = current->prev;
    else
        printf("Nothing to undo\n");
}
```

```

void redo() {
    if (current->next)
        current = current->next;
    else
        printf("Nothing to redo\n");
}

void show(char *label) {
    printf("%s: \"%s\"\n", label, current->text);
}

int main() {
    doAction("");
    doAction("Hello");
    doAction("Hello World");
    doAction("Hello World!");
    show("Typed");
    undo();
    show("Undo");
    undo();
    show("Undo");
    redo();
    show("Redo");
    doAction("Hello C");
    show("New edit");
    redo();
    undo();
    show("Undo");
    return 0;
}

```

Output:

```

Typed      : "Hello World!"
Undo       : "Hello World"
Undo       : "Hello"
Redo       : "Hello World"
New edit   : "Hello C"
Nothing to redo
Undo       : "Hello World"

```

2. Given two sorted lists L1 and L2, write a procedure to compute $L1 \cap L2$ and $L1 \cup L2$ using basic list operations.

Code:

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

```

```

Node* append(Node *tail, int val) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->next = NULL;
    tail->next = n;
    return n;
}

```

```

Node* insertEnd(Node *head, int val) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->next = NULL;
    if (head == NULL)
        return n;
    Node *t = head;
    while (t->next)
        t = t->next;
    t->next = n;
    return head;
}

```

```

Node* intersection(Node *a, Node *b) {
    Node dummy;
    Node *tail = &dummy;
    dummy.next = NULL;
    while (a && b) {
        if (a->data == b->data) {
            tail = append(tail, a->data);
            a = a->next;
            b = b->next;
        } else if (a->data < b->data)
            a = a->next;
        else
            b = b->next;
    }
    return dummy.next;
}

```

```

Node* unionList(Node *a, Node *b) {
    Node dummy;
    Node *tail = &dummy;
    dummy.next = NULL;
    while (a && b) {
        if (a->data == b->data) {
            tail = append(tail, a->data);
            a = a->next;
            b = b->next;
        } else if (a->data < b->data) {
            tail = append(tail, a->data);
            a = a->next;
        } else {
            tail = append(tail, b->data);
            b = b->next;
        }
    }
}

```

```

    }
    while (a) {
        tail = append(tail, a->data);
        a = a->next;
    }
    while (b) {
        tail = append(tail, b->data);
        b = b->next;
    }
    return dummy.next;
}

void display(char *label, Node *h) {
    printf("%s: ", label);
    while (h) {
        printf("%d ", h->data);
        h = h->next;
    }
    printf("\n");
}

int main() {
    Node *L1 = NULL, *L2 = NULL;
    int a[] = {1, 3, 5, 7, 9, 11};
    int b[] = {3, 4, 5, 6, 11, 12};
    for (int i = 0; i < 6; i++) {
        L1 = insertEnd(L1, a[i]);
        L2 = insertEnd(L2, b[i]);
    }
    display("L1", L1);
    display("L2", L2);
    display("L1 intersect L2", intersection(L1, L2));
    display("L1 union L2", unionList(L1, L2));
    return 0;
}

```

Sample Output:

```

L1: 1 3 5 7 9 11
L2: 3 4 5 6 11 12
L1 intersect L2: 3 5 11
L1 union L2: 1 3 4 5 6 7 9 11 12

```

3. Swap two elements by adjusting only the pointers (and not the data) using a doubly linked list.

Code:

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;

```

```

    struct Node *prev, *next;
} Node;

Node* insertEnd(Node *head, int val) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->next = NULL;
    if (!head) {
        n->prev = NULL;
        return n;
    }
    Node *t = head;
    while (t->next)
        t = t->next;
    t->next = n;
    n->prev = t;
    return head;
}

Node* find(Node *head, int x) {
    while (head && head->data != x)
        head = head->next;
    return head;
}

void swapNodes(Node **head, int x, int y) {
    if (x == y)
        return;
    Node *a = find(*head, x), *b = find(*head, y);
    if (!a || !b) {
        printf("Element not found\n");
        return;
    }
    Node *aPrev = a->prev, *aNext = a->next;
    Node *bPrev = b->prev, *bNext = b->next;

    if (aNext == b) {
        a->prev = b; a->next = bNext;
        b->prev = aPrev; b->next = a;
        if (aPrev) aPrev->next = b;
        if (bNext) bNext->prev = a;
    } else if (bNext == a) {
        b->prev = a; b->next = aNext;
        a->prev = bPrev; a->next = b;
        if (bPrev) bPrev->next = a;
        if (aNext) aNext->prev = b;
    } else {
        a->prev = bPrev; a->next = bNext;
        b->prev = aPrev; b->next = aNext;
        if (aPrev) aPrev->next = b;
        if (aNext) aNext->prev = b;
        if (bPrev) bPrev->next = a;
        if (bNext) bNext->prev = a;
    }
}

```

```

        if (*head == a) *head = b;
        else if (*head == b) *head = a;
    }

void display(Node *head) {
    while (head) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main() {
    Node *head = NULL;
    for (int i = 1; i <= 6; i++)
        head = insertEnd(head, i * 10);
    printf("Original list: ");
    display(head);
    swapNodes(&head, 20, 50);
    printf("Swap 20 and 50 (non-adjacent): ");
    display(head);
    swapNodes(&head, 30, 40);
    printf("Swap 30 and 40 (adjacent): ");
    display(head);
    swapNodes(&head, 10, 60);
    printf("Swap 10 and 60 (head and tail): ");
    display(head);
    return 0;
}

```

Sample Output:

```

Original list: 10 20 30 40 50 60
Swap 20 and 50 (non-adjacent): 10 50 30 40 20 60
Swap 30 and 40 (adjacent): 10 50 40 30 20 60
Swap 10 and 60 (head and tail): 60 50 40 30 20 10

```

4. Write an array implementation of self-adjusting lists. A self-adjusting list is like a regular list, except that all insertions are performed at the front, and when an element is accessed by a find, it is moved to the front of the list without changing the relative order of the other items.

Code:

```

#include <stdio.h>
#define MAX 100

typedef struct {
    int arr[MAX];
    int size;
} List;

void insert(List *l, int val) {

```

```

        if (l->size == MAX) {
            printf("List full\n");
            return;
        }
        for (int i = l->size; i > 0; i--)
            l->arr[i] = l->arr[i - 1];
        l->arr[0] = val;
        l->size++;
    }

int find(List *l, int val) {
    for (int i = 0; i < l->size; i++) {
        if (l->arr[i] == val) {
            for (int j = i; j > 0; j--)
                l->arr[j] = l->arr[j - 1];
            l->arr[0] = val;
            return 1;
        }
    }
    return 0;
}

void display(List *l) {
    for (int i = 0; i < l->size; i++)
        printf("%d ", l->arr[i]);
    printf("\n");
}

int main() {
    List l;
    l.size = 0;
    insert(&l, 10);
    insert(&l, 20);
    insert(&l, 30);
    insert(&l, 40);
    printf("After inserting 10, 20, 30, 40: ");
    display(&l);
    find(&l, 20);
    printf("After find(20): ");
    display(&l);
    find(&l, 40);
    printf("After find(40): ");
    display(&l);
    find(&l, 10);
    printf("After find(10): ");
    display(&l);
    if (!find(&l, 99))
        printf("find(99): not found, list unchanged\n");
    return 0;
}

```

Sample Output:

After inserting 10, 20, 30, 40: 40 30 20 10

```
After find(20): 20 40 30 10
After find(40): 40 20 30 10
After find(10): 10 40 20 30
find(99): not found, list unchanged
```

5. Write a C program to reverse a given doubly linked list.

Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *prev, *next;
} Node;

Node* insertEnd(Node *head, int val) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->next = NULL;
    if (!head) {
        n->prev = NULL;
        return n;
    }
    Node *t = head;
    while (t->next)
        t = t->next;
    t->next = n;
    n->prev = t;
    return head;
}

Node* reverse(Node *head) {
    Node *temp = NULL, *cur = head;
    while (cur) {
        temp = cur->prev;
        cur->prev = cur->next;
        cur->next = temp;
        cur = cur->prev;
    }
    if (temp)
        head = temp->prev;
    return head;
}

void display(Node *head) {
    while (head) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}
```

```

}

int main() {
    Node *head = NULL;
    for (int i = 1; i <= 5; i++)
        head = insertEnd(head, i * 10);
    printf("Original list: ");
    display(head);
    head = reverse(head);
    printf("Reversed list: ");
    display(head);
    return 0;
}

```

Sample Output:

```

Original list: 10 20 30 40 50
Reversed list: 50 40 30 20 10

```

6. Write a C program to delete the middle element of a DLL.

Code:

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *prev, *next;
} Node;

Node* insertEnd(Node *head, int val) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->next = NULL;
    if (!head) {
        n->prev = NULL;
        return n;
    }
    Node *t = head;
    while (t->next)
        t = t->next;
    t->next = n;
    n->prev = t;
    return head;
}

Node* deleteMiddle(Node *head) {
    if (!head)
        return NULL;
    Node *slow = head, *fast = head;
    while (fast && fast->next) {
        fast = fast->next->next;
    }
    if (fast == head)
        return head->next;
    if (fast == head->next)
        return head;
    slow->next = fast->next;
    fast->prev = slow;
    return head;
}

```

```

        slow = slow->next;
    }
    if (slow->prev)
        slow->prev->next = slow->next;
    else
        head = slow->next;
    if (slow->next)
        slow->next->prev = slow->prev;
    printf("Deleted middle element: %d\n", slow->data);
    free(slow);
    return head;
}

void display(Node *head) {
    while (head) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main() {
    Node *head = NULL;
    for (int i = 1; i <= 5; i++)
        head = insertEnd(head, i * 10);
    printf("Original list: ");
    display(head);
    head = deleteMiddle(head);
    printf("After deletion: ");
    display(head);

    head = insertEnd(head, 60);
    printf("\nList with even count: ");
    display(head);
    head = deleteMiddle(head);
    printf("After deletion: ");
    display(head);
    return 0;
}

```

Sample Output:

```

Original list: 10 20 30 40 50
Deleted middle element: 30
After deletion: 10 20 40 50

```

```

List with even count: 10 20 40 50 60
Deleted middle element: 40
After deletion: 10 20 50 60

```

7. Remove duplicate elements from a (i) sorted DLL (ii) unsorted DLL.

Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *prev, *next;
} Node;

Node* insertEnd(Node *head, int val) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->next = NULL;
    if (!head) {
        n->prev = NULL;
        return n;
    }
    Node *t = head;
    while (t->next)
        t = t->next;
    t->next = n;
    n->prev = t;
    return head;
}

Node* removeNode(Node *head, Node *del) {
    if (del->prev)
        del->prev->next = del->next;
    else
        head = del->next;
    if (del->next)
        del->next->prev = del->prev;
    free(del);
    return head;
}

Node* removeDupSorted(Node *head) {
    Node *cur = head;
    while (cur && cur->next) {
        if (cur->data == cur->next->data)
            head = removeNode(head, cur->next);
        else
            cur = cur->next;
    }
    return head;
}

Node* removeDupUnsorted(Node *head) {
    for (Node *cur = head; cur; cur = cur->next) {
        Node *p = cur->next;
        while (p) {
            Node *nxt = p->next;
            if (p->data == cur->data)
```

```

        head = removeNode(head, p);
        p = nxt;
    }
}
return head;
}

void display(Node *head) {
    while (head) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main() {
    int s[] = {1, 1, 2, 3, 3, 3, 4, 5, 5};
    int u[] = {4, 2, 4, 1, 2, 5, 1, 3};
    Node *sorted = NULL, *unsorted = NULL;
    for (int i = 0; i < 9; i++)
        sorted = insertEnd(sorted, s[i]);
    for (int i = 0; i < 8; i++)
        unsorted = insertEnd(unsorted, u[i]);

    printf("Sorted DLL before:  ");
    display(sorted);
    sorted = removeDupSorted(sorted);
    printf("Sorted DLL after:  ");
    display(sorted);

    printf("Unsorted DLL before: ");
    display(unsorted);
    unsorted = removeDupUnsorted(unsorted);
    printf("Unsorted DLL after:  ");
    display(unsorted);
    return 0;
}

```

Sample Output:

```

Sorted DLL before:  1 1 2 3 3 3 4 5 5
Sorted DLL after:   1 2 3 4 5
Unsorted DLL before: 4 2 4 1 2 5 1 3
Unsorted DLL after:  4 2 1 5 3

```

8. Calculate the frequency count of a given integer in a circular DLL.

Code:

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {

```

```

    int data;
    struct Node *prev, *next;
} Node;

Node* insertEnd(Node *head, int val) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    if (!head) {
        n->next = n->prev = n;
        return n;
    }
    Node *last = head->prev;
    n->next = head;
    n->prev = last;
    last->next = n;
    head->prev = n;
    return head;
}

int frequency(Node *head, int key) {
    if (!head)
        return 0;
    int count = 0;
    Node *cur = head;
    do {
        if (cur->data == key)
            count++;
        cur = cur->next;
    } while (cur != head);
    return count;
}

int main() {
    int arr[] = {5, 3, 5, 7, 3, 5, 9};
    Node *head = NULL;
    for (int i = 0; i < 7; i++)
        head = insertEnd(head, arr[i]);

    printf("Circular DLL: 5 3 5 7 3 5 9\n");
    printf("Frequency of 5 = %d\n", frequency(head, 5));
    printf("Frequency of 3 = %d\n", frequency(head, 3));
    printf("Frequency of 9 = %d\n", frequency(head, 9));
    printf("Frequency of 4 = %d\n", frequency(head, 4));
    return 0;
}

```

Sample Output:

```

Circular DLL: 5 3 5 7 3 5 9
Frequency of 5 = 3
Frequency of 3 = 2
Frequency of 9 = 1
Frequency of 4 = 0

```

9. Search for a particular element in a binary tree.

Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;

Node* newNode(int val) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->left = n->right = NULL;
    return n;
}

int search(Node *root, int key) {
    if (!root)
        return 0;
    if (root->data == key)
        return 1;
    return search(root->left, key) || search(root->right, key);
}

int main() {
    Node *root = newNode(1);
    root->left = newNode(2);
    root->right = newNode(3);
    root->left->left = newNode(4);
    root->left->right = newNode(5);
    root->right->right = newNode(6);

    int keys[] = {5, 6, 9};
    for (int i = 0; i < 3; i++) {
        if (search(root, keys[i]))
            printf("%d found in the tree\n", keys[i]);
        else
            printf("%d not found in the tree\n", keys[i]);
    }
    return 0;
}
```

Sample Output:

```
5 found in the tree
6 found in the tree
9 not found in the tree
```

10. Calculate the height or depth of a tree and print it.

Code:

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;

Node* newNode(int val) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->left = n->right = NULL;
    return n;
}

int height(Node *root) {
    if (!root)
        return -1;
    int lh = height(root->left);
    int rh = height(root->right);
    return (lh > rh ? lh : rh) + 1;
}

int main() {
    Node *root = newNode(1);
    root->left = newNode(2);
    root->right = newNode(3);
    root->left->left = newNode(4);
    root->left->right = newNode(5);
    root->left->left->left = newNode(6);

    printf("Height of the tree = %d\n", height(root));
    printf("Height of left subtree = %d\n", height(root->left));
    printf("Height of right subtree = %d\n", height(root->right));
    return 0;
}

```

Sample Output:

```

Height of the tree = 3
Height of left subtree = 2
Height of right subtree = 0

```

11. Given a binary tree, convert it to a doubly linked list keeping the same order (inorder traversal).**Code:**

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {

```

```

    int data;
    struct Node *left, *right;
} Node;

Node* newNode(int val) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->left = n->right = NULL;
    return n;
}

Node *prevNode = NULL, *headNode = NULL;

void toDLL(Node *root) {
    if (!root)
        return;
    toDLL(root->left);
    if (!prevNode)
        headNode = root;
    else {
        root->left = prevNode;
        prevNode->right = root;
    }
    prevNode = root;
    toDLL(root->right);
}

int main() {
    Node *root = newNode(10);
    root->left = newNode(12);
    root->right = newNode(15);
    root->left->left = newNode(25);
    root->left->right = newNode(30);
    root->right->left = newNode(36);

    toDLL(root);

    printf("DLL (forward): ");
    Node *t = headNode, *last = NULL;
    while (t) {
        printf("%d ", t->data);
        last = t;
        t = t->right;
    }
    printf("\nDLL (backward): ");
    while (last) {
        printf("%d ", last->data);
        last = last->left;
    }
    printf("\n");
    return 0;
}

```

Sample Output:

```
DLL (forward): 25 12 30 10 36 15
DLL (backward): 15 36 10 30 12 25
```

12. Given a binary tree, the task is to connect the nodes that are at the same level using next right pointers.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100

typedef struct Node {
    int data;
    struct Node *left, *right, *next;
} Node;

Node* newNode(int val) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->left = n->right = n->next = NULL;
    return n;
}

void connect(Node *root) {
    if (!root)
        return;
    Node *queue[MAX];
    int front = 0, rear = 0;
    queue[rear++] = root;
    while (front < rear) {
        int size = rear - front;
        for (int i = 0; i < size; i++) {
            Node *cur = queue[front++];
            cur->next = (i < size - 1) ? queue[front] : NULL;
            if (cur->left)
                queue[rear++] = cur->left;
            if (cur->right)
                queue[rear++] = cur->right;
        }
    }
}

void printLevels(Node *root) {
    Node *levelStart = root;
    while (levelStart) {
        Node *cur = levelStart, *nextStart = NULL;
        while (cur) {
            printf("%d -> ", cur->data);
            if (!nextStart)
                nextStart = cur->left ? cur->left : cur->right;
            cur = cur->next;
        }
        printf("\n");
        levelStart = nextStart;
    }
}
```

```

        cur = cur->next;
    }
    printf("NULL\n");
    levelStart = nextStart;
}
}

int main() {
    Node *root = newNode(1);
    root->left = newNode(2);
    root->right = newNode(3);
    root->left->left = newNode(4);
    root->left->right = newNode(5);
    root->right->right = newNode(6);
    root->left->left->left = newNode(7);

    connect(root);
    printf("Next right pointers level by level:\n");
    printLevels(root);
    return 0;
}

```

Sample Output:

```

Next right pointers level by level:
1 -> NULL
2 -> 3 -> NULL
4 -> 5 -> 6 -> NULL
7 -> NULL

```

13. Implement a priority queue using a binary search tree and perform the operation insert.

Code:

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int priority;
    struct Node *left, *right;
} Node;

Node* newNode(int p) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->priority = p;
    n->left = n->right = NULL;
    return n;
}

Node* insert(Node *root, int p) {
    if (!root)
        return newNode(p);

```

```

        if (p < root->priority)
            root->left = insert(root->left, p);
        else
            root->right = insert(root->right, p);
        return root;
    }

Node* extractMin(Node *root, int *minVal) {
    if (!root->left) {
        *minVal = root->priority;
        Node *r = root->right;
        free(root);
        return r;
    }
    root->left = extractMin(root->left, minVal);
    return root;
}

void inorder(Node *root) {
    if (!root)
        return;
    inorder(root->left);
    printf("%d ", root->priority);
    inorder(root->right);
}

int main() {
    Node *pq = NULL;
    int vals[] = {40, 15, 60, 10, 25, 50, 5};
    for (int i = 0; i < 7; i++) {
        pq = insert(pq, vals[i]);
        printf("Inserted %2d -> queue (in priority order): ", vals[i]);
        inorder(pq);
        printf("\n");
    }
    int m;
    pq = extractMin(pq, &m);
    printf("\nHighest priority element removed: %d\n", m);
    pq = extractMin(pq, &m);
    printf("Next highest priority element removed: %d\n", m);
    printf("Remaining queue: ");
    inorder(pq);
    printf("\n");
    return 0;
}

```

Sample Output:

```

Inserted 40 -> queue (in priority order): 40
Inserted 15 -> queue (in priority order): 15 40
Inserted 60 -> queue (in priority order): 15 40 60
Inserted 10 -> queue (in priority order): 10 15 40 60
Inserted 25 -> queue (in priority order): 10 15 25 40 60
Inserted 50 -> queue (in priority order): 10 15 25 40 50 60

```

Inserted 5 -> queue (in priority order): 5 10 15 25 40 50 60

Highest priority element removed: 5

Next highest priority element removed: 10

Remaining queue: 15 25 40 50 60

14. In a given binary search tree, write routines to delete a node when it has no children, one child and two children.

Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;

Node* newNode(int val) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->left = n->right = NULL;
    return n;
}

Node* insert(Node *root, int val) {
    if (!root)
        return newNode(val);
    if (val < root->data)
        root->left = insert(root->left, val);
    else
        root->right = insert(root->right, val);
    return root;
}

Node* minNode(Node *n) {
    while (n->left)
        n = n->left;
    return n;
}

Node* deleteNode(Node *root, int key) {
    if (!root) {
        printf("%d not found\n", key);
        return NULL;
    }
    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else {

```

```

        if (!root->left && !root->right) {
            free(root);
            return NULL;
        } else if (!root->left) {
            Node *t = root->right;
            free(root);
            return t;
        } else if (!root->right) {
            Node *t = root->left;
            free(root);
            return t;
        } else {
            Node *s = minNode(root->right);
            root->data = s->data;
            root->right = deleteNode(root->right, s->data);
        }
    }
    return root;
}

void inorder(Node *root) {
    if (!root)
        return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

int main() {
    Node *root = NULL;
    int vals[] = {50, 30, 70, 20, 40, 60, 80, 65};
    for (int i = 0; i < 8; i++)
        root = insert(root, vals[i]);
    printf("Inorder before deletion:      ");
    inorder(root);
    printf("\n");

    root = deleteNode(root, 20);
    printf("After deleting 20 (no child):   ");
    inorder(root);
    printf("\n");

    root = deleteNode(root, 60);
    printf("After deleting 60 (one child):  ");
    inorder(root);
    printf("\n");

    root = deleteNode(root, 50);
    printf("After deleting 50 (two children): ");
    inorder(root);
    printf("\n");
    printf("New root = %d\n", root->data);
    return 0;
}

```

Sample Output:

```
Inorder before deletion:      20 30 40 50 60 65 70 80
After deleting 20 (no child):  30 40 50 60 65 70 80
After deleting 60 (one child): 30 40 50 65 70 80
After deleting 50 (two children): 30 40 65 70 80
New root = 65
```

15. Convert the given postfix expression to an expression tree.**Code:**

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
#define MAX 100

typedef struct Node {
    char data;
    struct Node *left, *right;
} Node;

Node* newNode(char c) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = c;
    n->left = n->right = NULL;
    return n;
}

Node* buildTree(char *postfix) {
    Node *stack[MAX];
    int top = -1;
    for (int i = 0; postfix[i]; i++) {
        char c = postfix[i];
        Node *n = newNode(c);
        if (!isalnum(c)) {
            n->right = stack[top--];
            n->left = stack[top--];
        }
        stack[++top] = n;
    }
    return stack[top];
}

void infix(Node *root) {
    if (!root)
        return;
    if (!isalnum(root->data))
        printf("(");
    infix(root->left);
    printf("%c", root->data);
```

```

        infix(root->right);
        if (!isalnum(root->data))
            printf(" ");
    }

void prefix(Node *root) {
    if (!root)
        return;
    printf("%c", root->data);
    prefix(root->left);
    prefix(root->right);
}

void postorder(Node *root) {
    if (!root)
        return;
    postorder(root->left);
    postorder(root->right);
    printf("%c", root->data);
}

int main() {
    char postfix[] = "ab+cd-*e/";
    Node *root = buildTree(postfix);
    printf("Postfix expression: %s\n", postfix);
    printf("Infix (inorder):   ");
    infix(root);
    printf("\nPrefix (preorder): ");
    prefix(root);
    printf("\nPostfix (postorder): ");
    postorder(root);
    printf("\nRoot of expression tree: %c\n", root->data);
    return 0;
}

```

Sample Output:

```

Postfix expression: ab+cd-*e/
Infix (inorder):   ((a+b)*(c-d))/e)
Prefix (preorder):  /*+ab-cde
Postfix (postorder): ab+cd-*e/
Root of expression tree: /

```

16. Construct a binary search tree using (i) Inorder node representation, (ii) Preorder and (iii) Postorder.

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

typedef struct Node {

```

```

    int data;
    struct Node *left, *right;
} Node;

Node* newNode(int val) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->left = n->right = NULL;
    return n;
}

Node* fromInorder(int in[], int lo, int hi) {
    if (lo > hi)
        return NULL;
    int mid = (lo + hi) / 2;
    Node *root = newNode(in[mid]);
    root->left = fromInorder(in, lo, mid - 1);
    root->right = fromInorder(in, mid + 1, hi);
    return root;
}

Node* fromPreorder(int pre[], int *idx, int n, int min, int max) {
    if (*idx >= n || pre[*idx] <= min || pre[*idx] >= max)
        return NULL;
    int val = pre[*idx++];
    Node *root = newNode(val);
    root->left = fromPreorder(pre, idx, n, min, val);
    root->right = fromPreorder(pre, idx, n, val, max);
    return root;
}

Node* fromPostorder(int post[], int *idx, int min, int max) {
    if (*idx < 0 || post[*idx] <= min || post[*idx] >= max)
        return NULL;
    int val = post[*idx--];
    Node *root = newNode(val);
    root->right = fromPostorder(post, idx, val, max);
    root->left = fromPostorder(post, idx, min, val);
    return root;
}

void inorder(Node *r) {
    if (r) { inorder(r->left); printf("%d ", r->data); inorder(r->right); }
}

void preorder(Node *r) {
    if (r) { printf("%d ", r->data); preorder(r->left); preorder(r->right); }
}

void postorder(Node *r) {
    if (r) { postorder(r->left); postorder(r->right); printf("%d ", r->data); }
}

int main() {
    int in[]    = {20, 30, 40, 50, 60, 70, 80};
    int pre[]   = {50, 30, 20, 40, 70, 60, 80};

```

```

int post[] = {20, 40, 30, 60, 80, 70, 50};
int n = 7, i;

Node *t1 = fromInorder(in, 0, n - 1);
printf("(i) BST from inorder\n    Preorder of result: ");
preorder(t1);

i = 0;
Node *t2 = fromPreorder(pre, &i, n, INT_MIN, INT_MAX);
printf("\n(ii) BST from preorder\n    Inorder of result: ");
inorder(t2);
printf("\n    Postorder of result: ");
postorder(t2);

i = n - 1;
Node *t3 = fromPostorder(post, &i, INT_MIN, INT_MAX);
printf("\n(iii) BST from postorder\n    Inorder of result: ");
inorder(t3);
printf("\n    Preorder of result: ");
preorder(t3);
printf("\n");
return 0;
}

```

Sample Output:

```

(i) BST from inorder
    Preorder of result:  50 30 20 40 70 60 80
(ii) BST from preorder
    Inorder of result:   20 30 40 50 60 70 80
    Postorder of result: 20 40 30 60 80 70 50
(iii) BST from postorder
    Inorder of result:   20 30 40 50 60 70 80
    Preorder of result:  50 30 20 40 70 60 80

```

17. Implement heap sort for a set of numbers. Input: 50 20 30 25 15 22 32 (n = 7). Heap: 50 25 32 20 15 22 30.

Code:

```

#include <stdio.h>

void heapify(int arr[], int n, int i) {
    int largest = i;
    int l = 2 * i + 1, r = 2 * i + 2;
    if (l < n && arr[l] > arr[largest])
        largest = l;
    if (r < n && arr[r] > arr[largest])
        largest = r;
    if (largest != i) {
        int t = arr[i];
        arr[i] = arr[largest];
        arr[largest] = t;
    }
}

```

```

        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    printf("Max heap:      ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    for (int i = n - 1; i > 0; i--) {
        int t = arr[0];
        arr[0] = arr[i];
        arr[i] = t;
        heapify(arr, i, 0);
    }
}

int main() {
    int arr[] = {50, 20, 30, 25, 15, 22, 32};
    int n = 7;
    printf("Input:          ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
    heapSort(arr, n);
    printf("Sorted array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
    return 0;
}

```

Sample Output:

```

Input:      50 20 30 25 15 22 32
Max heap:   50 25 32 20 15 22 30
Sorted array: 15 20 22 25 30 32 50

```